

Sztuczna Inteligencja – laboratorium nr 5

1. LLM – Large Language Models

To nic innego jak zaawansowane głębokie sieci neuronowe (Model) stworzone do interpretacji i generowania języka ludzkiego (Language) na podstawie obszernych (Large) zbiorów danych. Modele LLM stanowią podgrupę modeli NLP (Natural Language Processing), będąc wydzieloną ze względu na swój rozmiar i złożoność.

Jedną z kluczowych decyzji, przed którą stają organizacje i indywidualni programiści, jest wybór pomiędzy rozwiązaniami lokalnymi i chmurowymi. Każde z nich ma swoje zalety i wady, a ich wybór zależy od wielu czynników, w tym od potrzeb, dostępnych zasobów i pożądaných wyników. W **Tabeli 1** przedstawione zostały zalety oraz wady rozwiązań w chmurze, natomiast w **Tabeli 2** rozwiązań instalowanych lokalnie.

Tabela 1.

LLMy w chmurze	
zalety	wady
• Łatwa skalowalność zasobów w zależności od potrzeb • Brak konieczności inwestowania w drogi sprzęt i infrastrukturę IT • Dostawcy modelu zajmują się utrzymaniem i aktualizacją infrastruktury	• Potencjalne obawy dotyczące bezpieczeństwa i prywatności danych przesyłanych do rozwiązań chmurowych • Dostępność zależna od wewnętrznych regulacji i polityk dostawcy • Filtrowanie treści, które mogą kolidować z modelem biznesowym (cyberbezpieczeństwo, medycyna i inne, gdzie model filtrujący treści uzna, że informacje mogą być wykorzystane w sposób niezgodny z polityką dostawcy) • Będąc dostawcą usługi opartej na zewnętrznym rozwiązaniu chmurowym, nie mamy żadnej kontroli nad kosztami (ich wzrostem) związanymi z wykorzystaniem zasobów usługi

Tabela 2.

LLMy lokalne	
zalety	wady
• Lokalne rozwiązania zapewniają pełną kontrolę nad danymi, co jest kluczowe w przypadku wrażliwych lub poufnych informacji • Możliwość dostosowania infrastruktury i oprogramowania do specyficznych wymagań projektu • Bezpośredni dostęp do sprzętu i oprogramowania bez względu na dostępność usług chmurowych • Możliwość wykorzystania dedykowanego, publicznie dostępnego modelu (lub własnego) do konkretnego zadania. Takie rozwiązanie często okazuje się daleko bardziej efektywne od dużego (i wymagającego więcej zasobów), ale bardzo uniwersalnego modelu • Możliwość stosowania lokalnych modeli tłumaczących przed przesłaniem ich do bazy lub głównego modelu • Brak ograniczeń przy tworzeniu baz wektorowych • Brak ograniczeń w wyborze ocenianych i nieocenianych modeli	• Wysokie koszty początkowe związane z zakupem, konfiguracją sprzętu oraz dedykowanego doradztwa • Konieczność zarządzania i utrzymania infrastruktury IT • Potencjalne ograniczenia w skalowaniu w porównaniu do elastyczności rozwiązań chmurowych

2. Ćwiczenie 1. Instalacja narzędzia do lokalnego uruchamiania modeli LLM

W tym ćwiczeniu będziemy używać narzędzia o nazwie **Ollama**, które eliminuje potrzebę wysyłania danych do zewnętrznych serwerów.

Oznacza to:

- **Izolację danych** - wrażliwe informacje o konfiguracji sieci pozostają w obrębie własnej infrastruktury
- **Niezależność** - możliwość korzystania z modeli AI bez polegania na zewnętrznych dostawcach, nawet w segmentach odizolowanych od Internetu
- **Kontrolę zasobów** - precyzyjne zarządzanie obciążeniem sieci i alokacją mocy obliczeniowej
- **Zgodność z politykami bezpieczeństwa** - spełnienie rygorystycznych wymogów bezpieczeństwa dla infrastruktury krytycznej

Narzędzie **Ollama** działa na zasadzie lekkiego kontenera, który zarządza modelami LLM i udostępnia je poprzez lokalny interfejs API, co ułatwia integrację z istniejącymi narzędziami sieciowymi.

Ściągnij i zainstaluj narzędzie z <https://ollama.com/>.

Następnie uruchom terminal i wpisz **ollama**. Jeśli narzędzie działa poprawnie powinno się wyświetlić:

```
D:\
λ ollama
Usage:
  ollama [flags]
  ollama [command]

Available Commands:
  serve    Start ollama
  create   Create a model from a Modelfile
  show     Show information for a model
  run      Run a model
  stop     Stop a running model
  pull     Pull a model from a registry
  push     Push a model to a registry
  list     List models
  ps       List running models
  cp       Copy a model
  rm       Remove a model
  help     Help about any command

Flags:
  -h, --help      help for ollama
  -v, --version   Show version information

Use "ollama [command] --help" for more information about a command.

D:\
λ
```

Następnie wpisz komendę `ollama run llama3.2`. Przy pierwszym uruchomieniu model o nazwie `llama3.2` zostanie ściągnięty do pamięci podręcznej narzędzia i uruchomiony. Możesz teraz wydawać polecenia wpisując prompty użytkownika, np.:

```
D:\
λ ollama run llama3.2
>>> who made you?
I was created by Meta AI, a company that specializes in natural language processing and artificial intelligence. My knowledge was built from a large corpus of text data, which I use to generate human-like responses to your questions and engage in conversation.

My training data includes a vast amount of text from various sources, including but not limited to:

1. Web pages
2. Books and articles
3. Conversational datasets
4. Product reviews

This training data allows me to understand the nuances of language, generate responses, and learn from interactions like ours.

However, I didn't exactly "make" myself. I'm more like a product of Meta AI's research and development efforts, designed to assist and communicate with humans in a helpful and informative way!

>>> | send a message (/? for help)
```

Wyjście z linii promptów poprzez `ctrl-d`

Sprawdzenie aktywnego modelu:

```
D:\
λ ollama ps
NAME                ID                SIZE    PROCESSOR    UNTIL
llama3.2:latest     a80c4f17acd5     4.0 GB    100% GPU     4 minutes from now

D:\
λ |
```

Lista innych modeli do przetestowania znajduje się na stronie <https://ollama.com/search>.

Niektóre z nich zostały dostarczone w kilku wersjach, różniących się ilością parametrów:

deepseek-r1

DeepSeek's first-generation of reasoning models with comparable performance to OpenAI-o1, including six dense models distilled from DeepSeek-R1 based on Llama and Qwen.

1.5b 7b 8b 14b 32b 70b 671b

↓ 39.7M Pulls 🔖 29 Tags ⌚ Updated 2 months ago

Jeśli chcemy użyć modelu, który ma wiele wersji możemy podać dodatkowo z ilu parametrów ma się składać. Poniżej wybraliśmy model z 8 miliardami parametrów:

```
D:\
λ ollama run deepseek-r1:8b
pulling manifest
pulling 6340dc3229b0... 100% ▓ 4.9 GB
pulling 369ca498f347... 100% ▓ 387 B
pulling 6e4c38e1172f... 100% ▓ 1.1 KB
pulling f4d24e9138dd... 100% ▓ 148 B
pulling 0cb05c6e4e02... 100% ▓ 487 B
verifying sha256 digest
writing manifest
success
>>> Send a message (/? for help)
```

Ma to oczywiście wpływ na wielkość pliku aby dopasować model do jakości posiadanego sprzętu. Modele te pracują w pamięci operacyjnej.

Przetestuj kilka modeli i porównaj ich jakość.

Najbardziej popularne wyświetlone są na stronie głównej ollamy:

Run [Llama 3.3](#), [DeepSeek-R1](#), [Phi-4](#), [Mistral](#),
[Gemma 3](#), and other models, locally.

Możesz sprawdzić aktualną listę dostępnych lokalnie modeli:

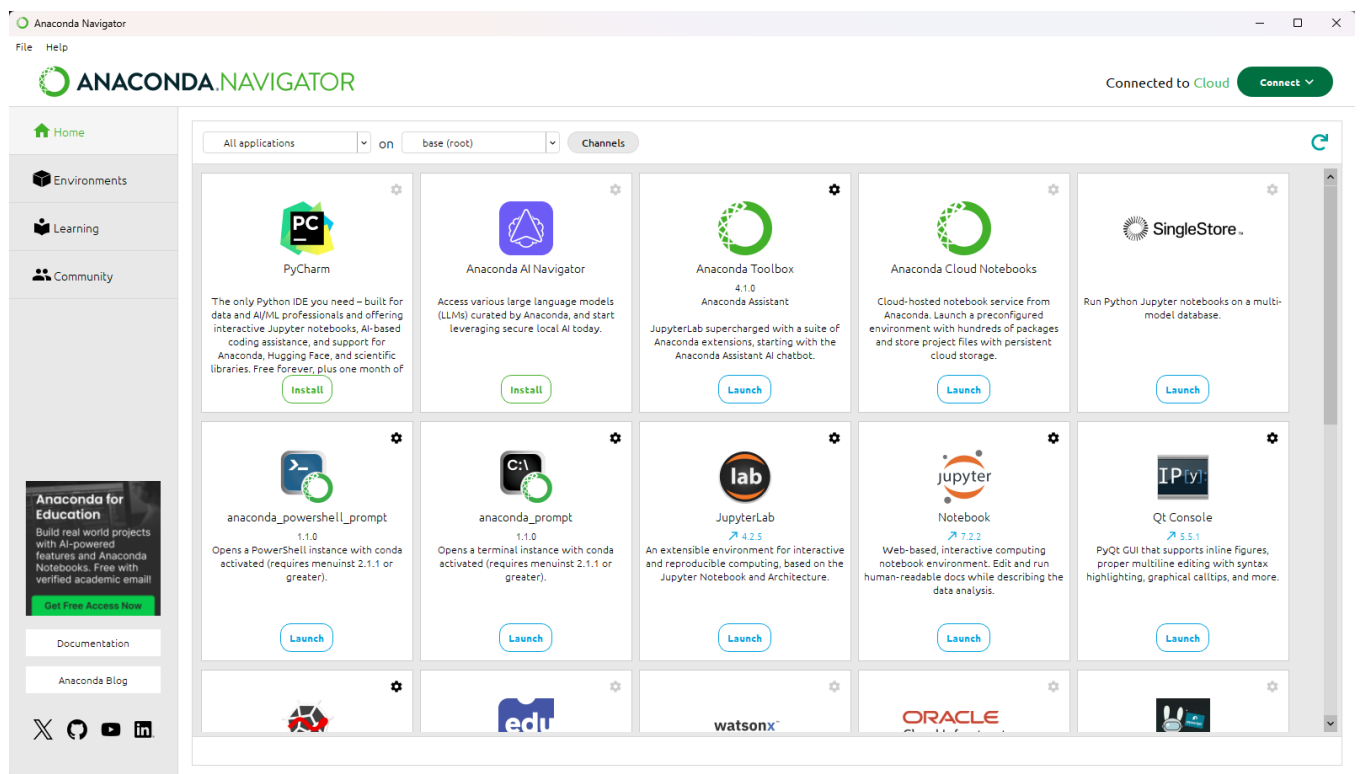
```
D:\
λ ollama list
```

NAME	ID	SIZE	MODIFIED
deepseek-r1:8b	28f8fd6cdc67	4.9 GB	2 hours ago
deepseek-r1:1.5b	a42b25d8c10a	1.1 GB	8 days ago
llama3.2:latest	a80c4f17acd5	2.0 GB	8 days ago
llama3.3:latest	a6eb4748fd29	42 GB	10 days ago
gemma2:27b	53261bc9c192	15 GB	10 days ago

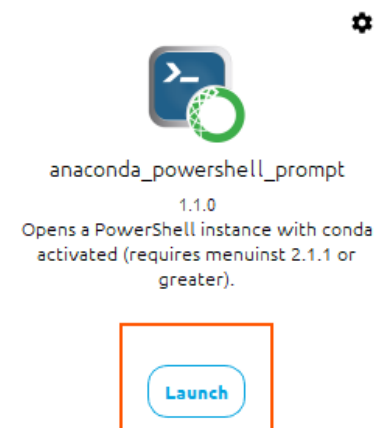
3. Ćwiczenie 2. Przygotowanie środowiska Anaconda do pracy z lokalnymi modelami.

Zainstaluj środowisko **Anaconda** ze strony: <https://www.anaconda.com/>.

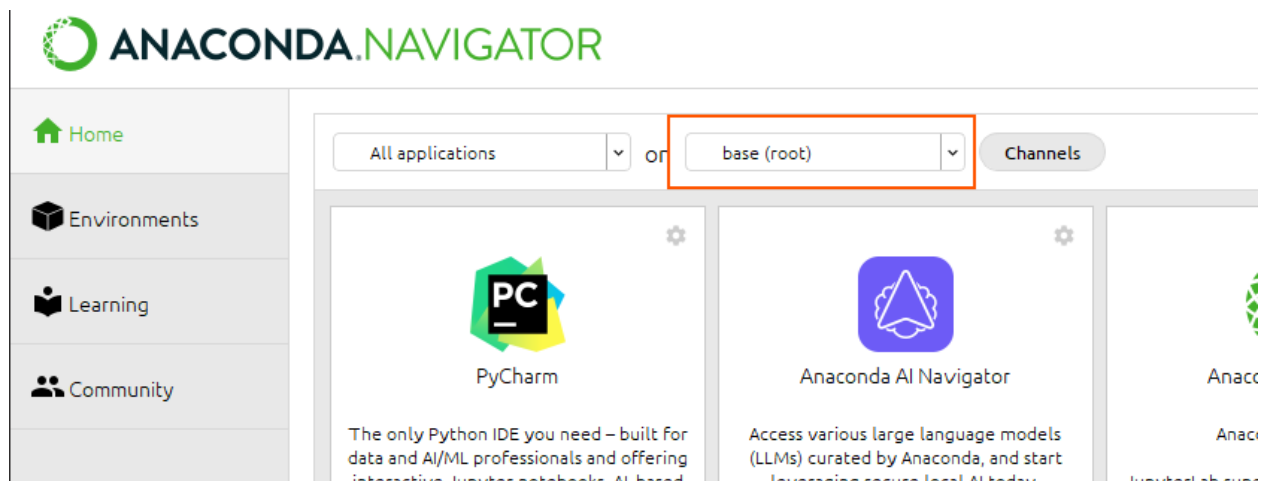
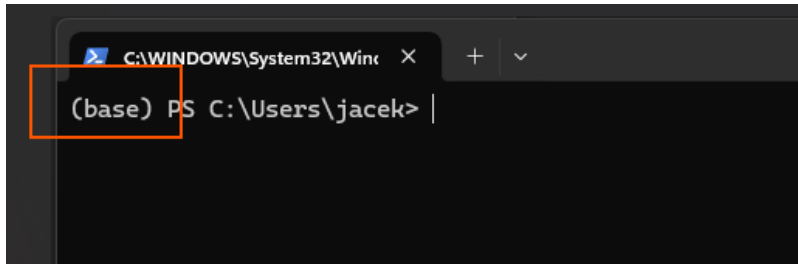
Po zainstalowaniu dostaniemy:



Uruchom następnie narzędzie:



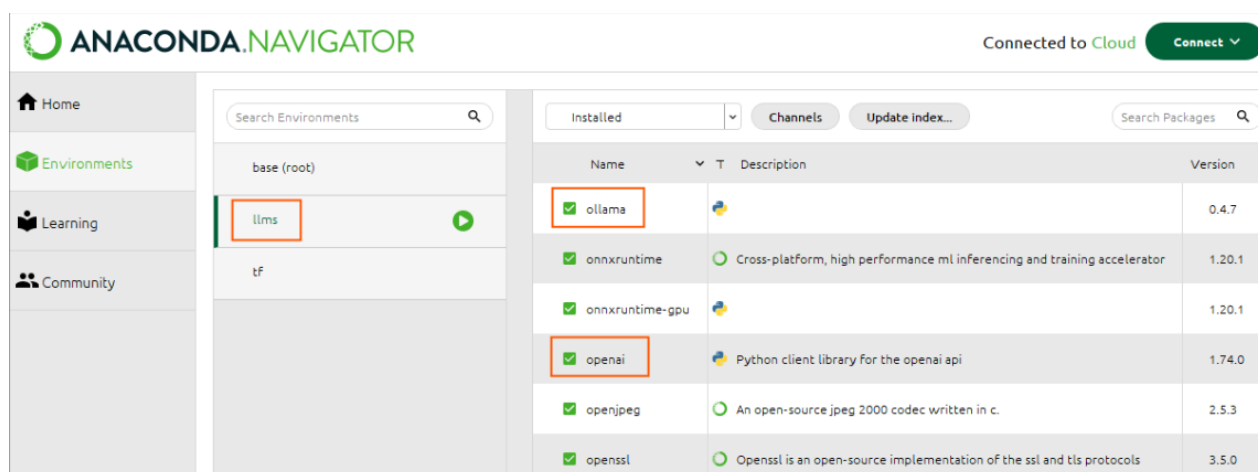
Dostaniemy terminal do pracy z podstawowym wirtualnym środowiskiem Anacondy:



Jednak to środowisko domyślnie nie ma wymaganych bibliotek i pakietów potrzebnych do pracy z modelami LLM.

Dlatego stworzymy nowe dedykowane.

Praca w wielu wirtualnych środowiskach pozwala nam na pracę przy wielu różnych projektach, które wymagają do pracy innych bibliotek, często w różnych wersjach:



Z moodle należy ściągnąć plik o nazwie [environment.yml](#) i skopiować go do nowo utworzonego katalogu w katalogu użytkownika np.:

```
(base) PS C:\Users\jacek> md llm

Directory: C:\Users\jacek

Mode                LastWriteTime         Length Name
----                -
d-----         25.04.2025        18:29             llm

(base) PS C:\Users\jacek> cd llm
(base) PS C:\Users\jacek\llm>
```

Plik ten pozwoli utworzyć nowe środowisko. Zawiera informacje o wszystkich potrzebnych zasobach do pracy z modelami LLM. Sprawdź aktualnie dostępne środowiska:

```
C:\WINDOWS\System32\Win... X + v
(base) PS C:\Users\jacek\llm> conda env list

# conda environments:
#
base                * C:\Users\jacek\anaconda3
llms                 C:\Users\jacek\anaconda3\envs\llms
tf                  C:\Users\jacek\anaconda3\envs\tf

(base) PS C:\Users\jacek\llm>
```

W Twoim pewnie będzie brakować środowiska [llms](#) więc je utworzymy. Wystarczy wpisać w terminal po skopiowaniu pliku:

```
C:\WINDOWS\System32\Win... X + v
(base) PS C:\Users\jacek\llm> conda env create -f environment.yml
```

Menadżer pakietów conda zainstaluje nam wszystkie zawarte w pliku biblioteki i stworzy nam nowe wirtualne środowisko.

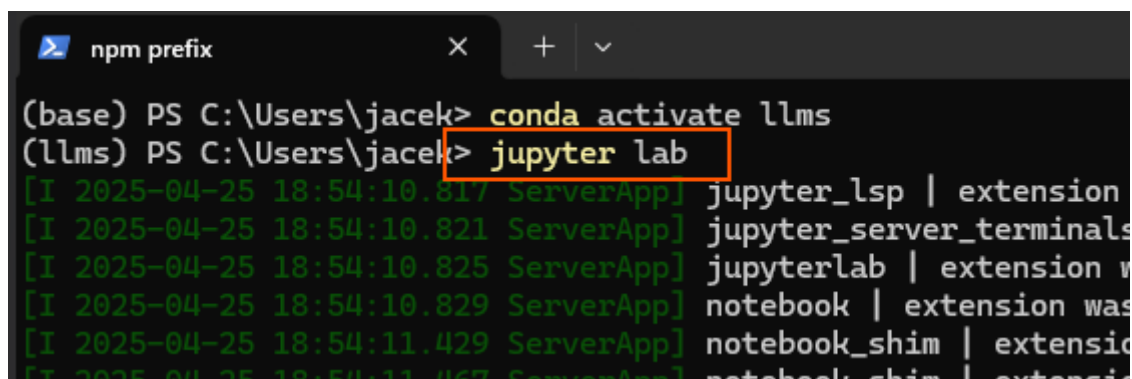
Środowisko następnie należy aktywować:

```
C:\WINDOWS\System32\Win... X + v
(base) PS C:\Users\jacek> conda activate llms
(llms) PS C:\Users\jacek>
```

Gotowe. Możemy teraz popracować z modelami LLM.

4. Ćwiczenie 3. Wykorzystanie modeli LLM w praktyce.

Uruchom teraz narzędzie JupyterLab:

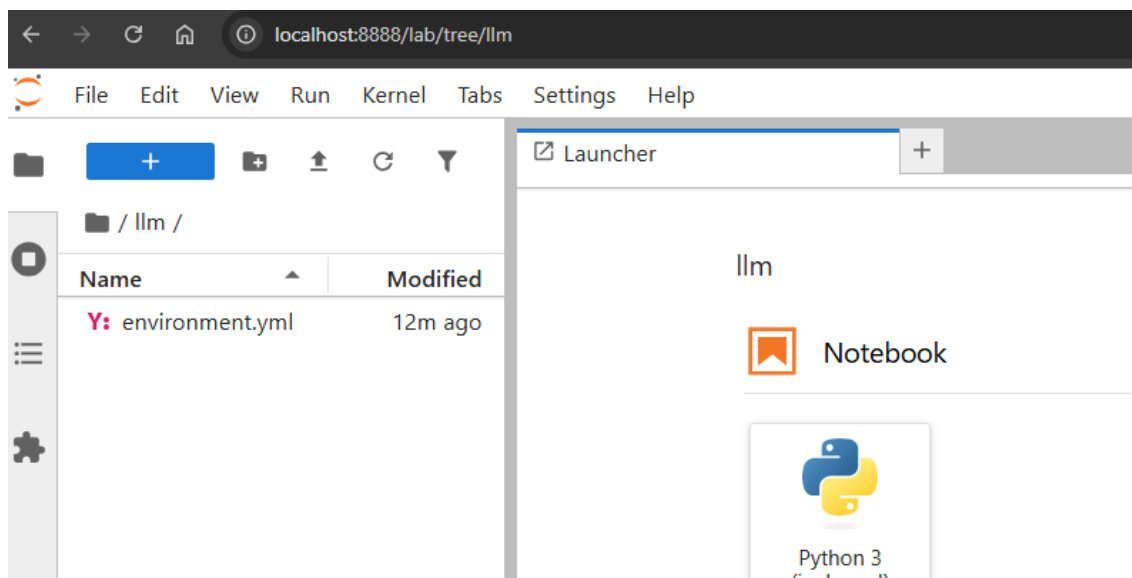


```
npm prefix
(base) PS C:\Users\jacek> conda activate llms
(llms) PS C:\Users\jacek> jupyter lab
[I 2025-04-25 18:54:10.817 ServerApp] jupyter_lsp | extension
[I 2025-04-25 18:54:10.821 ServerApp] jupyter_server_terminals
[I 2025-04-25 18:54:10.825 ServerApp] jupyterlab | extension w
[I 2025-04-25 18:54:10.829 ServerApp] notebook | extension was
[I 2025-04-25 18:54:11.429 ServerApp] notebook_shim | extensio
[I 2025-04-25 18:54:11.467 ServerApp] notebook_shim | extensio
```

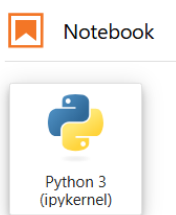
Wykorzystajmy więc model LLM w praktyce.

Naszym celem będzie zbudowanie narzędzia, któremu podamy adres URL, a ono odpowie podsumowaniem treści strony spod tego adresu.

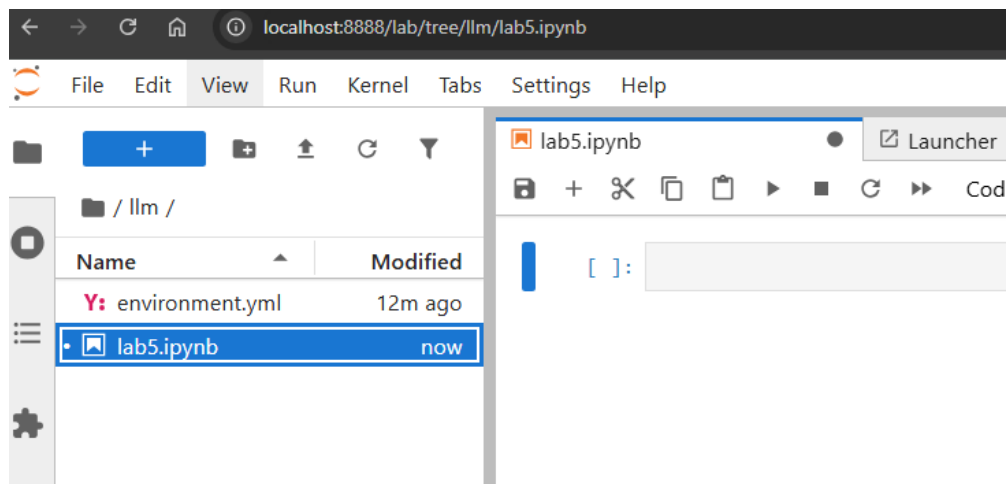
Pracujemy podobnie jak w Colab z blokami kodu w Python, tylko z użyciem narzędzia lokalnego JupyterLab, które powinno uruchomić się bezpośrednio w przeglądarce:



Klikamy:



i tworzymy nowy projekt, plik z rozszerzeniem `.ipynb`:



Na początku importujemy potrzebne biblioteki.

Pierwszy blok kodu:

```
import os
import requests
from dotenv import load_dotenv
from bs4 import BeautifulSoup
from IPython.display import Markdown, display
import ollama
```

Przetestujmy czy działa `ollama`. Testuj na małych modelach ze względu na czas oczekiwania na odpowiedź.

Drugi blok kodu:

```
message = "Tell me something about transformer technology"
response = ollama.chat(model='llama3.2', messages=[{"role": "user", "content": message}])
print(response['message']['content'])
```

Transformer technology, also known as electromagnetic induction, is a fundamental principle

Teraz zbudujemy prostą klasę, która będzie pobierała treści z wybranej strony.

Jest to tzw. `web scraper`.

Trzeci blok kodu

```
# Niektóre strony internetowe wymagają użycia odpowiednich nagłówków podczas ich pobierania:
headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/117.0.0.0 Safari/537.36"
}

# Klasa reprezentująca stronę internetową, która zostanie pobrana do podsumowania.
class Website:
    def __init__(self, url):
        self.url = url
        response = requests.get(url, headers=headers)
        # Użyjemy biblioteki BeautifulSoup do zbudowania obiektu z treścią
        soup = BeautifulSoup(response.content, 'html.parser')
        self.title = soup.title.string if soup.title else "No title found"
        for irrelevant in soup.body(["script", "style", "img", "input", "link"]):
            irrelevant.decompose()
        self.text = soup.body.get_text(separator="\n", strip=True)
```

I sprawdzimy czy działa.

Czwarty blok kodu:

```
web = Website("https://pl.wikipedia.org/wiki/Du%C5%BCy_model_j%C4%99zykowy")
print("Title: ", web.title)
print("text: ", web.text)
```

Przygotujemy teraz model do pracy. Modele takie jak GPT i jemu podobne zostały wytrenowane do odbierania instrukcji (**promptów**) w określony sposób. Oczekują one otrzymania:

- **promptu systemowego**, który mówi im, jakie zadanie wykonują i jakiego sposobu odpowiedzi model powinien użyć
- **promptu użytkownika** czyli inicjatora rozmowy, na który model powinien odpowiedzieć

Przygotujmy najpierw prompt systemowy.

Piąty blok kodu:

```
system_prompt = "You are an assistant that analyzes the contents of a website \
and provides a short summary, ignoring text that might be navigation related. \
Respond in markdown."
```

Możesz poeksperymentować tutaj. Zmień później ostatnią linię na np.: „**Respond in markdown in Polish**”

Teraz funkcja, która pisze prompt użytkownika proszący o podsumowanie strony internetowej.

Szósty blok kodu:

```
def user_prompt_for(website):
    user_prompt = f"You are looking at a website titled {website.title}"
    user_prompt += "\nThe contents of this website is as follows; \
please provide a short summary of this website in markdown. \
If it includes news or announcements, then summarize these too.\n\n"
    user_prompt += website.text
    return user_prompt
```

Instrukcje dla API modelu są przekazywane w tablicy w odpowiedniej strukturze.

Siódmy blok kodu:

```
def messages_for(website):
    return [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt_for(website)}
    ]
```

Jeśli chcesz zobaczyć jak wygląda taka wiadomość przesyłana do modelu, użyj:

```
messages_for(web)
```

```
[{'role': 'system',
  'content': 'You are an assistant that
d. Respond in markdown.'},
 {'role': 'user',
  'content': 'You are looking at a webs:
```

Użyjemy teraz modelu do podsumowania strony. Najpierw funkcja podsumowująca.

Ósmy blok kodu:

```
MODEL='llama3.2'
def summarize(url):
    website = Website(url)
    response = ollama.chat(
        model=MODEL,
        messages=messages_for(website)
    )
    return response['message']['content']
```

I test funkcji:

```
summarize("https://wp.pl")
```

Aby wykorzystać to, że model odpowiada w formacie „markdown” – tak mu w końcu kazaliśmy, przygotujemy funkcję, która to wykorzystuje:

Dziewiąty blok kodu:

```
def display_summary(url):  
    summary = summarize(url)  
    display(Markdown(summary))
```

I ostatnia linia naszego projektu:

```
display_summary("https://wp.pl")
```

Summary of Wirtualna Polska website

Overview ¶

The website is a Polish online news portal, providing information on various topics.

News and Announcements

Gotowe.

Jako zadanie przetestuj odpowiedzi różnych modeli i każ w prompcie systemowym odpowiadać w języku polskim.